十五届国赛_代码润色版

```c
#include <STC15F2K60S2.H>
#include "stdio.h"
#include "string.h"
#include "led.h"
#include "init.h"
#include "seg.h"
#include "key.h"
#include "iic.h"
#include "ultrasound.h"
#include "uart.h"
#include "math.h"

#define PI 3.14

// 频率测量
idata unsigned int freq = 0;
idata unsigned int Time_1S = 0;

// 串口
idata unsigned char Uart_Rx_Flag = 0;
idata unsigned char Uart_Rx_Index = 0;
pdata unsigned char Uart_Rx_Buf[10] =
{0,0,0,0,0,0,0,0,0,0};
idata unsigned char Uart_Rx_Tick = 0;
// 数码管
idata unsigned char Seg_Pos = 0;
pdata unsigned char Seg_Buf[8] = {10,10,10,10,10,10,10,10};
idata unsigned char Seg_Disp_Mode = 0;      // 0-坐标 1-速度
2-参数
idata unsigned char seg_slow_down;
// 状态机（空闲=0，运行=1，等待=2）
```

```
idata unsigned char State_Disp_Mode = 0;

// 按键
idata unsigned char Key_Val = 0;
idata unsigned char Key_Old = 0;
idata unsigned char Key_Down = 0;
idata unsigned char Key_Up = 0;
idata unsigned char key_slow_down;
// LED
pdata unsigned char ucled[8] = {0,0,0,0,0,0,0,0};
idata bit Led_Flash_Flag = 0;
idata unsigned char Led_Flash_Tick = 0;
idata unsigned int Time_3s = 0;
idata bit Reach_Flag = 0;
idata bit L3_Flag = 0;

// AD 光敏（光敏地址0x41，夜暗标志）
idata bit Dark_Work = 0;
idata unsigned char ad_slow_down;
// 超声波
idata unsigned char Barrier_Distance = 0;
idata unsigned char us_slow_down;
// 运动参数
pdata unsigned int Destination[2] = {0,0};       // 目的坐标
pdata unsigned int Current_Location[2] = {0,0}; // 当前位置
idata unsigned int Speed_10x = 0;                // 速度x10
idata unsigned char Para_R_10x = 10;             // Rx10
(1.0)
idata char Para_B = 0;                          // B
idata unsigned char Para_R_CT_10x = 0;          // 参数设置临
时R
idata char Para_CT_B = 0;                       // 参数设置临时B
idata bit Para_Setting_Mode = 0;                // 0-R 1-B
idata bit Gotit_Flag = 0;                       // 已接收目的坐标

// 运动模型变量（线性插值方案，避免逐帧sqrt）
```

```
pdata float speed_x = 0, speed_y = 0;                // 速度分量
pdata unsigned int Start_Location[2] = {0, 0};      // 运动起始位置
pdata unsigned int Movement_Time = 0;                // 运动累计时间（0.1s单位）
idata unsigned char cnt_movement_time = 0;           // ISR内100ms计数器
idata bit Speed_Update_Flag = 0;                     // 速度更新标志

idata unsigned char ctrl_slow_down;

/* ========== 刷新速度值（用当前R/B/freq计算Speed_10x）========== */
void Refresh_Speed()
{
    float temp_speed_10x;
    temp_speed_10x = ((float)(PI * (Para_R_10x / 10.0) * freq) / 100.0f + Para_B) * 10;
    Speed_10x = (temp_speed_10x < 0) ? 0 : (unsigned int)temp_speed_10x;
}

/* ========== 重置运动状态（锚定起点+重置时间+重算速度分量）========== */
void Reset_Movement()
{
    float dx_f, dy_f, dist;

    Start_Location[0] = Current_Location[0];
    Start_Location[1] = Current_Location[1];
    Movement_Time = 0;
    cnt_movement_time = 0;

    dx_f = (float)Destination[0] - (float)Current_Location[0];
```

```
    dy_f = (float)Destination[1] -
(float)Current_Location[1];
    dist = sqrt(dx_f * dx_f + dy_f * dy_f);
    if (dist > 0)
    {
        speed_x = ((float)Speed_10x / 10.0f) * dx_f / dist;
        speed_y = ((float)Speed_10x / 10.0f) * dy_f / dist;
    }
    else
    {
        speed_x = 0;
        speed_y = 0;
    }
}

/* ========== 控制函数（线性插值，200ms调度）========== */
void Ctrl_Proc()
{
    if(ctrl_slow_down<200) return;
    ctrl_slow_down=0;
    if (State_Disp_Mode == 1)
    {
        float new_x, new_y;
        float t_sec = Movement_Time * 0.1f;

        // 障碍物保护
        if (Barrier_Distance <= 30)
        {
            State_Disp_Mode = 2;
            return;
        }

        // 线性插值计算当前位置
        new_x = (float)Start_Location[0] + speed_x * t_sec;
        new_y = (float)Start_Location[1] + speed_y * t_sec;
```

```
            // 到达检测
            if (speed_x >= 0)
            {
                if (new_x >= (float)Destination[0]) new_x =
(float)Destination[0];
            }
            else
            {
                if (new_x <= (float)Destination[0]) new_x =
(float)Destination[0];
            }
            if (speed_y >= 0)
            {
                if (new_y >= (float)Destination[1]) new_y =
(float)Destination[1];
            }
            else
            {
                if (new_y <= (float)Destination[1]) new_y =
(float)Destination[1];
            }

            Current_Location[0] = (unsigned int)(new_x + 0.5f);
            Current_Location[1] = (unsigned int)(new_y + 0.5f);

            // 两轴都到达目的地
            if (Current_Location[0] == Destination[0] &&
Current_Location[1] == Destination[1])
            {
                State_Disp_Mode = 0;
                Reach_Flag = 1;
                Time_3s = 0;
            }
        }
}
```

```c
/* ========== 速度计算任务（主循环）========== */
void Speed_Calc_Proc()
{
    if (Speed_Update_Flag)
    {
            unsigned int Old_Speed_10x;
            Speed_Update_Flag = 0;

            Old_Speed_10x = Speed_10x;
            Refresh_Speed();

            // 运行态且速度实际变化时：锚定当前位置，重置时间，重算速度分量
            if (State_Disp_Mode == 1 && Speed_10x != Old_Speed_10x)
            {
                    Reset_Movement();
            }
    }
}

/* ========== 按键处理 ========== */
void Key_Proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Val ^ Key_Old);
    Key_Up = ~Key_Val & (Key_Val ^ Key_Old);
    Key_Old = Key_Val;

    // S4：启动/暂停/恢复
    if (Key_Down == 4)
    {
        if (State_Disp_Mode == 0 && Gotit_Flag == 1)    // 空闲且有目标
```

```
            {
                Gotit_Flag = 0;
                Refresh_Speed();
                Reset_Movement();
                State_Disp_Mode = 1;
            }
            else if (State_Disp_Mode == 2 && Barrier_Distance
>= 30) // 等待且障碍消失
            {
                Refresh_Speed();
                Reset_Movement();
                State_Disp_Mode = 1;
            }
            else if (State_Disp_Mode == 1)                    //
运行 → 等待
            {
                State_Disp_Mode = 2;
            }
        }

        // S5：重置当前位置（空闲态有效）
        if (Key_Down == 5)
        {
            if (State_Disp_Mode == 0)
            {
                Current_Location[0] = 0;
                Current_Location[1] = 0;
            }
        }

        // S8：界面切换
        if (Key_Down == 8)
        {
            switch (Seg_Disp_Mode)
            {
            case 0:
```

```
            Seg_Disp_Mode = 1;
            break;
        case 1:
            Para_CT_B = Para_B;
            Para_R_CT_10x = Para_R_10x;
            Seg_Disp_Mode = 2;
            Para_Setting_Mode = 0;
            break;
        case 2:
            Para_B = Para_CT_B;
            Para_R_10x = Para_R_CT_10x;
            Seg_Disp_Mode = 0;
            break;
        }
    }

    // 参数设置下的按键
    if (Seg_Disp_Mode == 2)
    {
        if (Key_Down == 9)
            Para_Setting_Mode = !Para_Setting_Mode;

        if (Key_Down == 12)
        {
            if (Para_Setting_Mode == 0)
            {
                if (Para_R_CT_10x < 20) Para_R_CT_10x++;
            }
            else
            {
                if (Para_CT_B < 90) Para_CT_B += 5;
            }
        }

        if (Key_Down == 13)
        {
```

```
            if (Para_Setting_Mode == 0)
            {
                if (Para_R_CT_10x > 10) Para_R_CT_10x--;
            }
            else
            {
                if (Para_CT_B > -90) Para_CT_B -= 5;
            }
        }
    }
}

/* ========== 数码管显示 ========== */
void Seg_Proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;

    switch (Seg_Disp_Mode)
    {
    case 0:  // 坐标界面
        Seg_Buf[0] = 11;
        if (State_Disp_Mode == 0)
        {
            Seg_Buf[1] = (Current_Location[0] >= 100) ? Current_Location[0] / 100 % 10 : 10;
            Seg_Buf[2] = (Current_Location[0] >= 10)  ? Current_Location[0] / 10 % 10 : 10;
            Seg_Buf[3] = Current_Location[0] % 10;
            Seg_Buf[4] = 14;
            Seg_Buf[5] = (Current_Location[1] >= 100) ? Current_Location[1] / 100 % 10 : 10;
            Seg_Buf[6] = (Current_Location[1] >= 10)  ? Current_Location[1] / 10 % 10 : 10;
            Seg_Buf[7] = Current_Location[1] % 10;
        }
```

```
        else
        {
            Seg_Buf[1] = (Destination[0] >= 100) ?
Destination[0] / 100 % 10 : 10;
            Seg_Buf[2] = (Destination[0] >= 10)  ?
Destination[0] / 10 % 10 : 10;
            Seg_Buf[3] = Destination[0] % 10;
            Seg_Buf[4] = 14;
            Seg_Buf[5] = (Destination[1] >= 100) ?
Destination[1] / 100 % 10 : 10;
            Seg_Buf[6] = (Destination[1] >= 10)  ?
Destination[1] / 10 % 10 : 10;
            Seg_Buf[7] = Destination[1] % 10;
        }
        break;

    case 1:  // 速度/障碍界面
        Seg_Buf[0] = 12;
        Seg_Buf[2] = 10;
        if (State_Disp_Mode == 1)
        {
            Seg_Buf[1] = 1;
            Seg_Buf[3] = (Speed_10x >= 10000) ? Speed_10x /
10000 % 10 : 10;
            Seg_Buf[4] = (Speed_10x >= 1000)  ? Speed_10x /
1000 % 10 : 10;
            Seg_Buf[5] = (Speed_10x >= 100)   ? Speed_10x /
100 % 10 : 10;
            Seg_Buf[6] = Speed_10x / 10 % 10 + ',';
            Seg_Buf[7] = Speed_10x % 10;
        }
        else if (State_Disp_Mode == 0)
        {
            Seg_Buf[1] = 2;
            Seg_Buf[3] = 14;
            Seg_Buf[4] = 14;
```

```
            Seg_Buf[5] = 14;
            Seg_Buf[6] = 14;
            Seg_Buf[7] = 14;
        }
        else
        {
            Seg_Buf[1] = 3;
            Seg_Buf[3] = (Barrier_Distance >= 10000) ?
Barrier_Distance / 10000 % 10 : 10;
            Seg_Buf[4] = (Barrier_Distance >= 1000)  ?
Barrier_Distance / 1000 % 10 : 10;
            Seg_Buf[5] = (Barrier_Distance >= 100)   ?
Barrier_Distance / 100 % 10 : 10;
            Seg_Buf[6] = (Barrier_Distance >= 10)    ?
Barrier_Distance / 10 % 10 : 10;
            Seg_Buf[7] = Barrier_Distance % 10;
        }
        break;

    case 2:  // 参数界面
        Seg_Buf[0] = 13;
        Seg_Buf[1] = 10;
        Seg_Buf[2] = (Para_R_CT_10x / 10 % 10) + ',';
        Seg_Buf[3] = Para_R_CT_10x % 10;
        Seg_Buf[4] = 10;

        if (Para_CT_B > 0)
        {
            Seg_Buf[5] = 10;
            Seg_Buf[6] = (Para_CT_B >= 10) ? Para_CT_B / 10
% 10 : 10;
            Seg_Buf[7] = Para_CT_B % 10;
        }
        else if (Para_CT_B < 0)
        {
            if ((-Para_CT_B) >= 10)
```

```
            {
                Seg_Buf[5] = 14;
                Seg_Buf[6] = (-Para_CT_B) / 10 % 10;
                Seg_Buf[7] = (-Para_CT_B) % 10;
            }
            else
            {
                Seg_Buf[5] = 10;
                Seg_Buf[6] = 14;
                Seg_Buf[7] = (-Para_CT_B) % 10;
            }
        }
        else
        {
            Seg_Buf[5] = 10;
            Seg_Buf[6] = 10;
            Seg_Buf[7] = 0;
        }
        break;
    }
}

/* ========== LED 与继电器 ========== */
void Led_Proc()
{
    switch (State_Disp_Mode)
    {
    case 0:
        ucled[0] = 0;
        ucled[1] = 0;
        Relay(0);
        break;
    case 1:
        ucled[0] = 1;
        ucled[1] = Dark_Work;
        Relay(1);
```

```
            break;
        case 2:
            ucled[0] = Led_Flash_Flag;
            ucled[1] = 0;
            Relay(0);
            break;
        }
        ucled[2] = L3_Flag;
}

void Get_AD()
{
    unsigned char temp_ad;
    if(ad_slow_down<100) return;
    ad_slow_down=0;
    temp_ad = Ad_Read(0x41);
    Dark_Work = (temp_ad <= 1.2 * 51);
}

/* ========== 超声波测距 ========== */
void Get_Distance()
{
    if(us_slow_down<100) return;
    us_slow_down=0;

    Barrier_Distance = Ut_Wave_Data();
    if (Barrier_Distance == 0)
        Barrier_Distance = 255;

    if (State_Disp_Mode == 1 && Barrier_Distance <= 30)
        State_Disp_Mode = 2;
}

/* ========== 串口通信 ========== */
void Uart_Proc()
{
```

```
    unsigned int x, y;
    int result,num;
    if (Uart_Rx_Index == 0) return;
    if (Uart_Rx_Tick >= 10)
    {
        Uart_Rx_Flag = 0;
        Uart_Rx_Tick = 0;

        if (Uart_Rx_Buf[0] == '(' &&
Uart_Rx_Buf[Uart_Rx_Index - 1] == ')')
        {
            result=sscanf(Uart_Rx_Buf, "(%u,%u)%n", &x,
&y,&num);
            if (result==2&&strlen(Uart_Rx_Buf)==num)
            {
                if (State_Disp_Mode == 0)
                {
                    printf("Got it");
                    Gotit_Flag = 1;
                    Destination[0] = x;
                    Destination[1] = y;
                }
                else if (State_Disp_Mode == 2)
                {
                    printf("Wait");
                }
                else
                {
                    printf("Busy");
                }
            }
            else
            {
                printf("Error");
            }
        }
```

```
        else if (strcmp(Uart_Rx_Buf, "?") == 0)
        {
            if (State_Disp_Mode == 0)
                printf("Idle");
            else if (State_Disp_Mode == 2)
                printf("Wait");
            else
                printf("Busy");
        }
        else if (strcmp(Uart_Rx_Buf, "#") == 0)
        {
            printf("(%u,%u)", Current_Location[0],
Current_Location[1]);
        }
        else
        {
            printf("Error");
        }

        memset(Uart_Rx_Buf, 0, 10);
        Uart_Rx_Index = 0;
    }
}

/* ========== 串口中断 ========== */
void Uart1_Isr(void) interrupt 4
{
    if (RI)
    {
        Uart_Rx_Flag = 1;
        Uart_Rx_Tick = 0;
        if (Uart_Rx_Index < 10)
        {
            Uart_Rx_Buf[Uart_Rx_Index++] = SBUF;
        }
        else
```

```
        {
            Uart_Rx_Index = 0;
            Uart_Rx_Flag = 0;
            Uart_Rx_Tick = 0;
            memset(Uart_Rx_Buf, 0, 10);
        }
        RI = 0;
    }
}

/* ========== 定时器0（外部计数，测频率）========== */
void Timer0_Init(void)
{
    AUXR &= 0x7F;
    TMOD &= 0xF0;
    TMOD |= 0x05;
    TL0 = 0x00;
    TH0 = 0x00;
    TF0 = 0;
    TR0 = 1;
}

/* ========== 定时器1（1ms 系统时钟）========== */
void Timer1_Init(void)
{
    AUXR &= 0xBF;
    TMOD &= 0x0F;
    TL1 = 0x18;
    TH1 = 0xFC;
    TF1 = 0;
    TR1 = 1;
    ET1 = 1;
    EA = 1;
}

void Timer1_Isr(void) interrupt 3
```

```
{
    key_slow_down++;
    seg_slow_down++;
    ctrl_slow_down++;
    us_slow_down++;
    ad_slow_down++;
    // 数码管动态扫描
    if (Seg_Buf[Seg_Pos] > 20)
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos] - ',', 1);
    else
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos], 0);
    Seg_Pos = (++Seg_Pos) % 8;

    // 等待态 LED 闪烁 (0.1s)
    if (State_Disp_Mode == 2)
    {
        if (++Led_Flash_Tick == 100)
        {
            Led_Flash_Tick = 0;
            Led_Flash_Flag = !Led_Flash_Flag;
        }
    }
    else
    {
        Led_Flash_Tick = 0;
        Led_Flash_Flag = 0;
    }

    // 到达后 L3 亮3秒
    if (Reach_Flag)
    {
        L3_Flag = 1;
        if (++Time_3s == 3000)
        {
            Time_3s = 0;
            L3_Flag = 0;
```

```
                Reach_Flag = 0;
            }
        }

        // 运动时间累加
        if (State_Disp_Mode == 1)
        {
            if (++cnt_movement_time == 100)
            {
                cnt_movement_time = 0;
                Movement_Time++;
            }
        }

        // 每秒读频率
        if (++Time_1S == 1000)
        {
            Time_1S = 0;
            freq = (TH0 << 8) | TL0;
            TH0 = TL0 = 0;
            Speed_Update_Flag = 1;
        }

        // 串口超时计时
        if (Uart_Rx_Flag) Uart_Rx_Tick++;
        Led_Disp(ucled);
    }

    /* ========== 主函数 ========== */
    void main()
    {
        System_Init();
        Ad_Read(0x41);
        Timer0_Init();
        Uart1_Init();
        Timer1_Init();
```

```
    while (1)
    {
        Ctrl_Proc();
        Speed_Calc_Proc();
        Key_Proc();
        Seg_Proc();
        Led_Proc();
        Get_AD();
        Get_Distance();
        Uart_Proc();
    }
}
```

Fence 1